



信息学奥林匹克竞赛

Olympiad in Informatics

数据结构——树状数组

陈 真

上海市实验学校

扫描线与数点问题

问题再现

【问题1】一个长度为 n 的序列 $a_1 \sim a_n$ ，每次给出一个 i ，求 $\sum a_i$ ，即求序列的前缀和。

【思路】累加求和，for一遍就可以啦！

【问题2】一个长度为 n 的序列 $a_1 \sim a_n$ ，修改某个元素的值，再次询问前缀和，假设有 n 次修改， m 次询问。

【思路】暴力求解的时间复杂度为 $O(nm)$ ，当 $n, m \geq 10000$ 时，就会超时。

【分析】暴力求解慢的原因在哪里？

就是每次求前缀和时，重复计算！

如果把前缀固定分解成**若干个区间**，每次修改某个元素的时候，只影响其所在区间，其它区间还是**保持原值**，就会提高求前缀和的效率。

现在的问题转化为：**如何划分区间？**

树状数组

固定区间划分

根据任意正整数的关于2的不重复次幂的唯一分解性质，即任意正整数只有唯一的二进制表示形式。

若一个正整数 x 可以被“二进制分解”为： $x = 2^{i_1} + 2^{i_2} + \dots + 2^{i_m}$

设 $i_1 > i_2 > \dots > i_m$ ，则区间 $[1, x]$ 可以分成 $O(\log x)$ 个小区间：

1. 长度为 2^{i_1} 的小区间 $[1, 2^{i_1}]$
2. 长度为 2^{i_2} 的小区间 $[2^{i_1} + 1, 2^{i_1} + 2^{i_2}]$
3. 长度为 2^{i_3} 的小区间 $[2^{i_1} + 2^{i_2} + 1, 2^{i_1} + 2^{i_2} + 2^{i_3}]$
-
- m. 长度为 2^{i_m} 的小区间

小区间的共同特点：若区间结尾为 r ，则区间长度就等于 r 的“二进制分解”下**最小的2的次幂**，也就是最后一个1的位置的权值，即 $\text{lowbit}(r)$ 。

例如： $x=7=2^2+2^1+2^0$ ，那么区间 $[1, 7]$ 就可以分成 $[1, 4]$ 、 $[5, 6]$ 、 $[7, 7]$ 三个小区间，长度分别为 $\text{lowbit}(4)=4$ 、 $\text{lowbit}(6)=2$ 、 $\text{lowbit}(7)=1$

因为每一个 lowbit (右端点)是只算了右端点二进制的权值(十进制)，相当于把前后的所有1都抛掉，当然就是本段的大小了。

树状数组

构造树状数组

根据描述，构造一个由原数组的某段区间和组成的一个新数组，这个区间是以某个数为右端点，区间大小为 **lowbit(右端点)**

新数组即为**树状数组**

设**树状数组**为 $c[i]$ ，**原数组**为 $a[i]$ ，则： $c[i] = a[i] + a[i-1] + \dots + a[i - \text{lowbit}(i) + 1]$ 。

树状数组前8个元素的构成：

$$c[1] = a[1]$$

$$c[2] = a[1] + a[2] = c[1] + a[2]$$

$$c[3] = a[3]$$

$$c[4] = a[1] + a[2] + a[3] + a[4] = c[2] + c[3] + a[4]$$

$$c[5] = a[5]$$

$$c[6] = a[5] + a[6] = c[5] + a[6]$$

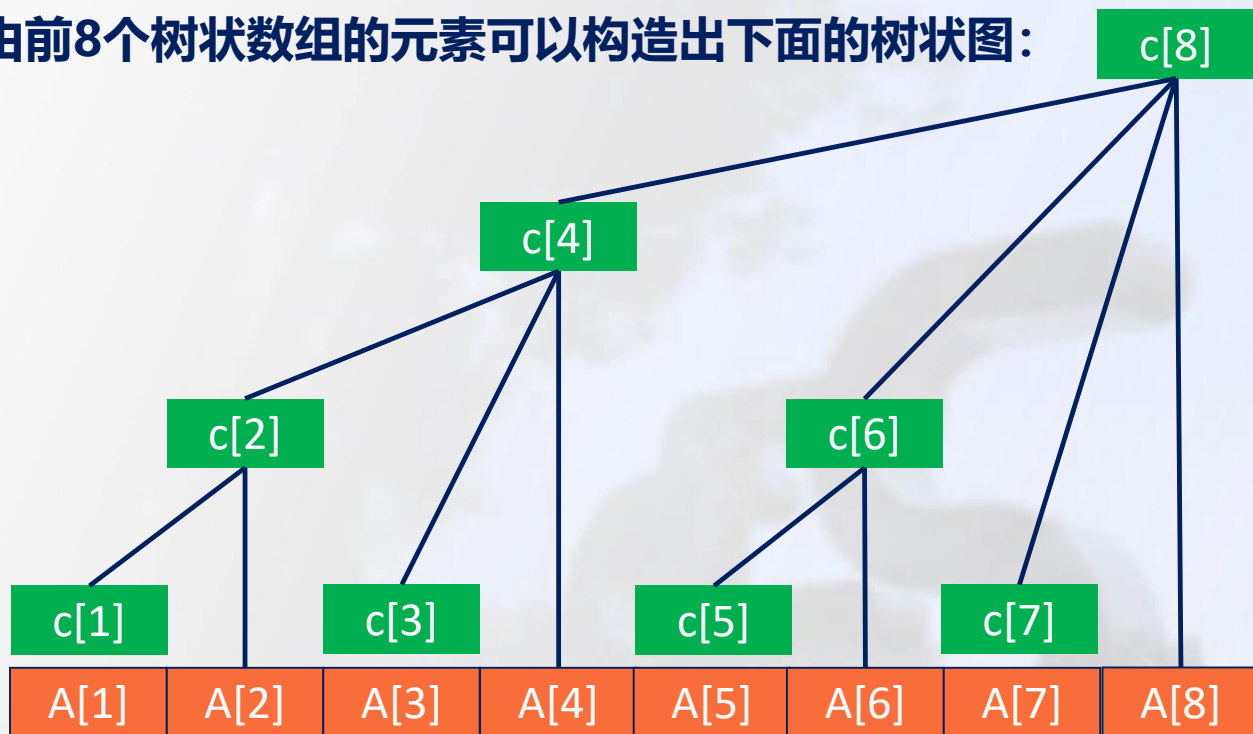
$$c[7] = a[7]$$

$$c[8] = a[1] + a[2] + a[3] + a[4] + a[5] + a[6] + a[7] + a[8] = c[4] + c[6] + c[7] + a[8]$$

树状数组

构造树状数组

由前8个树状数组的元素可以构造出下面的树状图：



根据树状数组的构造方式，可以得出如下性质：

1. 树状数组中某个节点*i*的父节点的下标为： $i + \text{lowbit}(i)$
2. 树状数组中某个节点*i*的兄弟(左边的)下标为： $i - \text{lowbit}(i)$

如何求lowbit()

用如下的函数求解lowbit(x)

```
int lowbit(int x){  
    return x & -x;  
}
```

为什么这样可以求lowbit(x)?

$x \& (-x)$ 是将x表示成二进制后，求出最后一个1的位置

例如： $14 \& (-14) = 2$

$$\begin{array}{r} 1110 \\ \& 0010 \\ \hline 0010 \end{array}$$

树状数组

单点修改

- 如果改变原数组的某一个元素值，称为单点修改，一般是指对这个元素进行加一个值val。
- 单点修改应该对涉及到该点的所有树状数组的点进行修改，也就是沿父节点一路往上，直到区间的最右端，时间复杂度为 $O(\log n)$ 。

```
void add(int x, int val){
    while(x <= n){
        c[x] += val;
        x += lowbit(x);
    }
}
```

询问前缀和

对于询问的前缀区间，从右端点开始，每次累加，每次移到左边兄弟的树状数组值，即每次下标减lowbit值。

```
int query(int x){
    int sum = 0;
    while(x){
        sum += c[x];
        x -= lowbit(x);
    }
}
```


树状数组

核心性质

树状数组的下标**必须从 1 开始**（由 lowbit 操作的特性决定的， $x=0$ 时 lowbit (0) 无意义），其所有操作都围绕**lowbit**展开。

回顾： $\text{lowbit}(x) = x \& -x$ ，作用是保留 x 的二进制中最右边的 1。

比如 $x=6$ (110)， $\text{lowbit}(6)=2$ (10)；

$x=8$ (1000)， $\text{lowbit}(8)=8$ (1000)。

性质 1：向上爬公式（找父节点）

节点编号为 x 时，父节点编号为 $x + \text{lowbit}(x)$ 。树状数组是一棵“不完全的树”，每个节点都有对应的父节点，修改操作时需要沿着父节点一路向上更新，父节点维护的区间包含当前节点。

性质 2：向前跳公式（找前一个相邻区间）

节点编号为 x 时，前一个相邻区间的编号为 $x - \text{lowbit}(x)$ 。查询前缀和 $[1, x]$ 时，需要通过这个公式不断向前跳，累加每个区间的和，直到跳至 0 为止。

性质 3：节点维护的区间范围

节点编号为 x 时，该节点维护的区间是 $[x - \text{lowbit}(x) + 1, x]$ 。树状数组的核心，每个节点都有明确的区间管辖范围，比如：

- $x=5$ (101)， $\text{lowbit}(5)=1$ ，维护区间 $[5, 5]$ ；
- $x=6$ (110)， $\text{lowbit}(6)=2$ ，维护区间 $[5, 6]$ ；
- $x=8$ (1000)， $\text{lowbit}(8)=8$ ，维护区间 $[1, 8]$ 。

树状数组

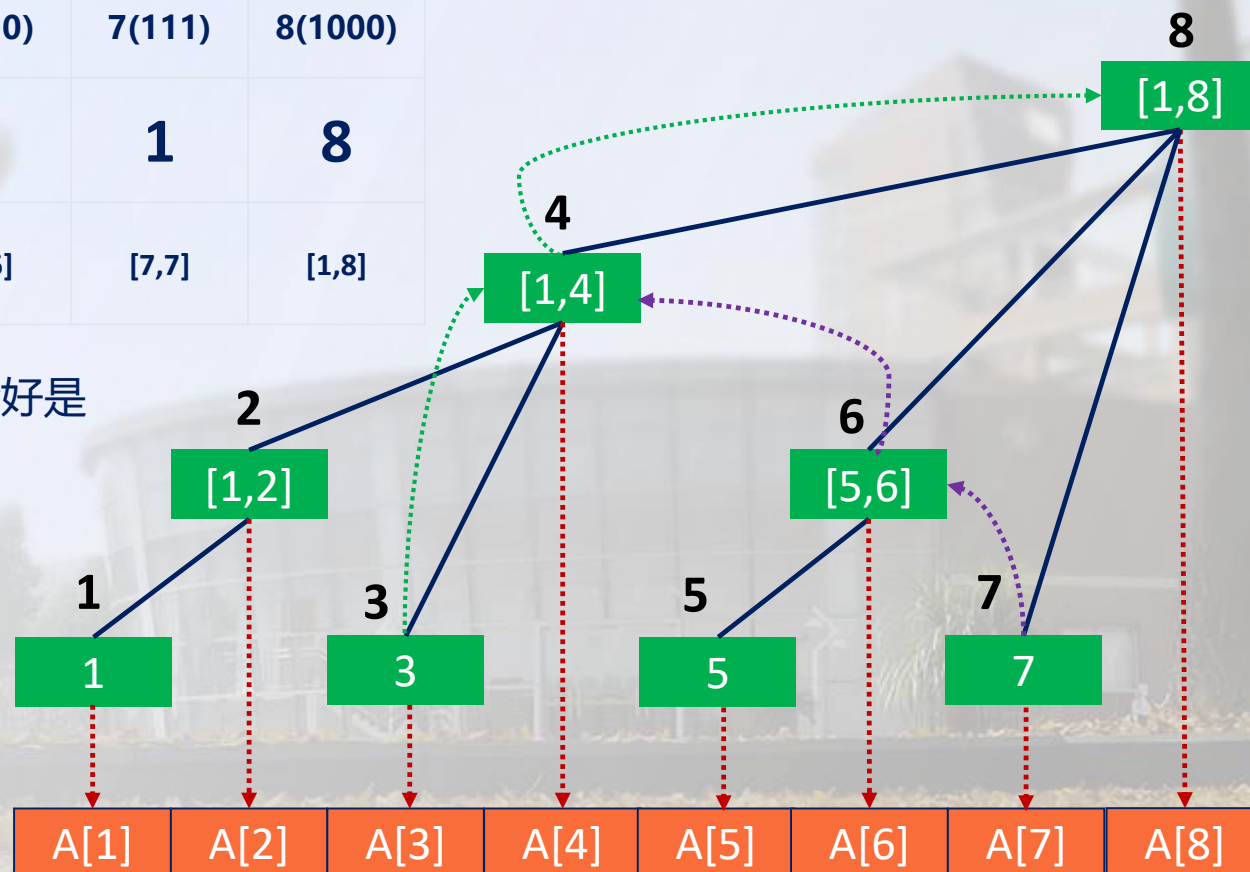
核心性质

结合性质，我们给出 $n=8$ 时树状数组的完整节点分布，每个节点的编号、lowbit 值、维护区间一目了然：

点 x	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)	8(1000)
$\text{lowbit}(x)$	1	2	1	4	1	2	1	8
维护区间	[1,1]	[1,2]	[3,3]	[1,4]	[5,5]	[5,6]	[7,7]	[1,8]

树状数组的每个节点都只维护一段连续的区间，且区间长度恰好是 $\text{lowbit}(x)$ 。

- 序列的单点修改 + 区间查询
- 区间修改 + 单点查询
- 区间修改 + 区间查询



树状数组

基础版：单点修改 + 区间查询

解决问题：给定一个长度为 n 的序列，执行 m 次操作，要么修改某个位置的数值，要么查询某个区间 $[l,r]$ 的和。

核心思路

【单点修改】 当位置 x 的数值增加 k 时，所有包含 x 的树状数组节点都需要更新，沿着父节点 $x += \text{lowbit}(x)$ 一路向上，直到超过 n ；

【区间查询】 利用前缀和思想，先查询 $[1,r]$ 的和，再查询 $[1,l-1]$ 的和，两者的差就是 $[l,r]$ 的和；

【树状数组创建】 初始时树状数组全为 0，读入序列的每个元素 $a[i]$ ，相当于对位置 i 执行单点修改（加 $a[i]$ ）。

```
8 // 单点修改：位置x增加k
9 void modify(int x, long long k) {
10     for(int i=x; i<=n; i+=lowbit(i)){
11         tree[i]+=k;
12     }
13 }
14 // 前缀和查询：查询[1,x]的和
15 long long query(int x){
16     long long res=0;
17     for (int i=x; i>0; i-=lowbit(i)){
18         res+=tree[i];
19     }
20     return res;
21 }
```

树状数组

进阶版 1：区间修改 + 单点查询

解决问题：问题变成将区间 $[x,y]$ 的所有数加 d ，查询某个位置 i 的数值，直接用基础版树状数组无法实现，此时需要结合**差分思想**。

回顾差分性质：

- 对原数组 a 的区间 $[x,y]$ 加 d ，等价于对差分数组 $diff$ 的 x 位置加 d ， $y+1$ 位置减 d ；
- 原数组的某个位置 i 的数值，等价于差分数组 $diff$ 的**前缀和** $[1,i]$ 。

区间修改 + 单点查询的问题可以转化为：

用树状数组维护**差分数组 $diff$** ，将原问题的区间修改变为树状数组的**两次单点修改**，原问题的单点查询变为树状数组的**前缀和查询**。

```
6 long long tree[N]; // 树状数组维护差分数组diff
7 void modify(int x, long long k){
8     for(int i=x; i<=n; i+=lowbit(i)){
9         tree[i]+=k;
10    }
11 }
12 // 查询差分数组的前缀和[1,x]，即原数组a[x]的值
13 long long query(int x){
14     long long res=0;
15     for(int i=x; i>0; i-=lowbit(i)){
16         res+=tree[i];
17     }
18     return res;
19 }
20 // 区间修改：原数组[a,b]的所有数加k
21 void interval_modify(int a, int b, long long k){
22     modify(a, k);
23     if(b+1<=n){
24         modify(b+1, -k);
25     }
26 }
```

树状数组

进阶版 2：区间修改 + 区间查询

解决问题：将区间 $[x,y]$ 的所有数加 k ，查询区间 $[l,r]$ 的和。该问题需要结合**差分 + 数学推导**，用**两个树状数组**维护不同的信息。

核心推导 设原数组为 a ，差分数组为 d ，满足 $a[i]=\sum_{j=1}^i d[j]$ 。
原数组的前缀和 $S[i] = a[1]+a[2]+\dots+a[i]$ ，
将 $a[j]$ 用差分数组表示并展开：

$$\begin{aligned} S[i] &= \sum_{j=1}^i \sum_{k=1}^j d[k] \\ &= d[1] \times i + d[2] \times (i-1) + d[3] \times (i-2) + \dots + d[i] \times 1 \\ &= i \times \sum_{k=1}^i d[k] - \sum_{k=1}^i d[k] \times (k-1) \end{aligned}$$

$$\text{sum1}[i] = \sum_{k=1}^i d[k]$$

$$\text{sum2}[i] = \sum_{k=1}^i d[k] \times (k-1)$$

原数组的前缀和 $S[i]=i \times \text{sum1}[i] - \text{sum2}[i]$ 。

原数组的区间和 $[1,r] = S[r] - S[1-1]$ ，只需用**两个树状数组**分别维护 $d[k]$ 和 $d[k] \times (k-1)$ ，可实现区间修改 + 区间查询。

树状数组

进阶版 2：区间修改 + 区间查询

解决问题：将区间 $[x,y]$ 的所有数加 k ，查询区间 $[l,r]$ 的和。该问题需要结合**差分 + 数学推导**，用**两个树状数组**维护不同的信息。

核心思路

1. 用树状数组 A 维护 $d[k]$ ，树状数组 B 维护 $d[k] \times (k-1)$;
2. 对原数组区间 $[x,y]$ 加 k ，等价于对差分数组 d 执行：
 $d[x]+k$ 、 $d[y+1]-k$ ，因此对 A 和 B 分别执行两次单点修改：
- ◆ $A.modify(x, k)$ 、 $A.modify(y+1, -k)$
- ◆ $B.modify(x, k \times (x-1))$ 、 $B.modify(y+1, -k \times y)$
3. 求原数组前缀和 $S[i]$ ： $i * A.query(i) - B.query(i)$;
4. 求原数组区间和 $[l,r]$ ： $S[r] - S[l-1]$ 。

```
6 struct BIT{
7     long long tree[N]; // 单点修改
8     void modify(int x, long long k){
9         for (int i=x; i<=n; i+=lowbit(i)){
10             tree[i]+=k;
11         }
12     }
13     long long query(int x){ // 前缀和查询
14         long long res=0;
15         for (int i=x; i>0; i-=lowbit(i)){
16             res+=tree[i];
17         }
18         return res;
19     }
20 } A, B;
21 // A维护d[k], B维护d[k]*(k-1)
22 // 原数组区间[x,y]加k
23 void interval_modify(int x, int y, long long k){
24     A.modify(x, k);
25     A.modify(y+1, -k);
26     B.modify(x, k*(x-1));
27     B.modify(y+1, -k*y);
28 }
29 // 求原数组前缀和[1,i]
30 long long prefix_sum(int i){
31     return (long long)i*A.query(i)-B.query(i);
32 }
33 // 求原数组区间和[l,r]
34 long long interval_query(int l, int r){
35     return prefix_sum(r)-prefix_sum(l-1);
36 }
```

树状数组(二维)

基础版：单点修改 + 子矩阵查询

解决问题：给定 $n \times m$ 的二维矩阵，执行若干操作，要么修改矩阵中位置 (x,y) 的数值，要么查询左上角为 $(x1,y1)$ 、右下角为 $(x2,y2)$ 的子矩阵的和。

核心思路：

单点修改：对位置 (x,y) 加 k ，需要对行和列分别执行向上爬操作，
行 $i += \text{lowbit}(i)$ ，列 $j += \text{lowbit}(j)$ ，直到超过 n 和 m ；

前缀和查询：查询从 $(1,1)$ 到 (x,y) 的前缀子矩阵和，行 $i -= \text{lowbit}(i)$ ，列 $j -= \text{lowbit}(j)$ ，累加所有节点的和；

子矩阵查询：利用二维前缀和思想，子矩阵 $(x1,y1)-(x2,y2)$ 的和为：

$$\text{sum}(x2,y2) - \text{sum}(x1-1,y2) - \text{sum}(x2,y1-1) + \text{sum}(x1-1,y1-1)$$

其中 $\text{sum}(x,y)$ 表示 $(1,1)$ 到 (x,y) 的前缀和。

```
7 long long tree[N][N];
8 // 单点修改: 位置(x,y)加k
9 void modify(int x,int y,long long k){
10     for(int i=x;i<=n;i+=lowbit(i)){
11         for(int j=y;j<=m;j+=lowbit(j)){
12             tree[i][j]+=k;
13         }
14     }
15 }
16 // 前缀和查询: (1,1)到(x,y)的和
17 long long query(int x,int y){
18     long long res=0;
19     for(int i=x;i>0;i-=lowbit(i)){
20         for(int j=y;j>0;j-=lowbit(j)){
21             res+=tree[i][j];
22         }
23     }
24     return res;
25 }
26 // 子矩阵查询: (x1,y1)到(x2,y2)的和
27 long long matrix_query(int x1,int y1,int x2,int y2){
28     return query(x2,y2)-query(x1-1,y2)-query(x2,y1-1)+query(x1-1,y1-1);
29 }
30 }
```

树状数组(二维)

进阶版：区间修改 + 单点查询

解决问题：对 $n \times m$ 矩阵的子矩阵 $(x1,y1)-(x2,y2)$ 的所有数加 d ，查询矩阵中某个位置 (x,y) 的数值。同样需要结合二维差分思想。

二维差分的核心性质

对原矩阵 a 的子矩阵 $(x1,y1)-(x2,y2)$ 加 d ，等价于对二维差分数组 $diff$ 执行四次单点修改：

$$\begin{aligned} diff[x1][y1] + &= d \\ diff[x1][y2 + 1] - &= d \\ diff[x2 + 1][y1] - &= d \\ diff[x2 + 1][y2 + 1] + &= d \end{aligned}$$

原矩阵的位置 (x,y) 的数值，等价于二维差分数组 $diff$ 的前缀和 $sum(1,1,x,y)$ 。

问题转化为：用二维树状数组维护二维差分数组 $diff$ ，原问题的区间修改变为树状数组的四次单点修改，原问题的单点查询变为树状数组的前缀和查询。

```
6 long long tree[N][N];
7 void modify(int x,int y,long long k) {
8     for(int i=x;i<=n;i+=lowbit(i)){
9         for(int j=y;j<=m;j+=lowbit(j)){
10             tree[i][j]+=k;
11         }
12     }
13 }
14 // 查询diff的前缀和，即原矩阵a[x][y]的值
15 long long query(int x,int y){
16     long long res=0;
17     for(int i=x;i>0;i-=lowbit(i)){
18         for(int j=y;j>0;j-=lowbit(j)){
19             res+=tree[i][j];
20         }
21     }
22     return res;
23 }
24 // 区间修改：子矩阵(x1,y1)-(x2,y2)加k
25 void matrix_modify(int x1,int y1,int x2,int y2,long long k){
26     modify(x1,y1,k);
27     if(y2+1<=m) modify(x1,y2+1,-k);
28     if(x2+1<=n) modify(x2+1,y1,-k);
29     if(x2+1<=n&& y2+1<=m) modify(x2+1,y2+1,k);
30 }
```


树状数组(二维)

进阶版：区间修改 + 子矩阵查询

二维树状数组的最高阶用法，解决问题：对矩阵的子矩阵 $(x1,y1)-(x2,y2)$ 加 k ，查询另一个子矩阵 $(a1,b1)-(a2,b2)$ 的和。该问题需要结合二维差分 + 数学推导，用四个二维树状数组维护不同的信息。

设二维原矩阵为 a ，二维差分数组为 d ，满足：

$$a[i][j] = \sum_{x=1}^i \sum_{y=1}^j d[x][y]$$

原矩阵的前缀和

$$s[i][j] = \sum_{x=1}^i \sum_{y=1}^j a[x][y]$$

$$S[i][j] = (i * j + i + j + 1) \times \sum d[x][y] - (j + 1) \times \sum d[x][y] \times x - (i + 1) \times \sum d[x][y] \times y + \sum d[x][y] \times x \times y$$

四个二维树状数组分别维护：

$d[x][y]$; $d[x][y] \times x$; $d[x][y] \times y$; $d[x][y] \times x \times y$

```
6 struct BIT2D {
7     long long tree[N][N];
8     void modify(int x,int y,long long k){
9         for(int i=x;i<=n;i+=lowbit(i)){
10             for(int j=y;j<=m;j+=lowbit(j)){
11                 tree[i][j]+=k;
12             }
13         }
14     }
15     long long query(int x,int y){
16         long long res=0;
17         for(int i=x;i>0;i-=lowbit(i)){
18             for(int j=y;j>0;j-=lowbit(j)){
19                 res+=tree[i][j];
20             }
21         }
22         return res;
23     }
24 }A, B, C, D;
25 // A: d[x][y]
26 // B: d[x][y] * x
27 // C: d[x][y] * y
28 // D: d[x][y] * x * y
29 // 二维差分的四次修改，同步更新四个树状数组
```

树状数组(二维)

进阶版：区间修改 + 子矩阵查询

二维树状数组的最高阶用法，解决问题：对矩阵的子矩阵 $(x1,y1)-(x2,y2)$ 加 k ，查询另一个子矩阵 $(a1,b1)-(a2,b2)$ 的和。该问题需要结合二维差分 + 数学推导，用四个二维树状数组维护不同的信息。

设二维原矩阵为 a ，二维差分数组为 d ，满足：

$$a[i][j] = \sum_{x=1}^i \sum_{y=1}^j d[x][y]$$

原矩阵的前缀和

$$s[i][j] = \sum_{x=1}^i \sum_{y=1}^j a[x][y]$$

$$S[i][j] = (i * j + i + j + 1) \times \sum d[x][y] - (j + 1) \times \sum d[x][y] \times x - (i + 1) \times \sum d[x][y] \times y + \sum d[x][y] \times x \times y$$

四个二维树状数组分别维护：

$d[x][y]$; $d[x][y] \times x$; $d[x][y] \times y$; $d[x][y] \times x \times y$

```
31 void add(int x,int y,long long k){
32     A.modify(x,y,k);
33     B.modify(x,y,k*x);
34     C.modify(x,y,k*y);
35     D.modify(x,y,k*x*y);
36 }
37 // 子矩阵(x1,y1)-(x2,y2)加k
38 void Modify(int x1,int y1,int x2,int y2,long long k) {
39     add(x1,y1,k);
40     add(x1,y2+1,-k);
41     add(x2+1,y1,-k);
42     add(x2+1,y2+1,k);
43 }
44 // 求原矩阵(1,1)到(x,y)的前缀和
45 long long Sum(int x,int y) {
46     long long res=(long long)(x*y+x+y+1)*A.query(x,y);
47     res-=(long long)(y+1)*B.query(x,y);
48     res-=(long long)(x+1)*C.query(x,y);
49     res+=D.query(x,y);
50     return res;
51 }
52 // 求原矩阵子矩阵(a1,b1)-(a2,b2)的和
53 long long Query(int a1,int b1,int a2,int b2) {
54     return Sum(a2,b2)-Sum(a1-1,b2)-Sum(a2,b1-1)+Sum(a1-1,b1-1);
55 }
```

总结归纳

一维树状数组

问题类型	核心思路	树状数组数量
单点修改 + 区间查询	直接维护原数组前缀和	1
区间修改 + 单点查询	维护差分数组，转化为单点操作	1
区间修改 + 区间查询	差分 + 数学推导，维护两个信息	2

二维树状数组

问题类型	核心思路	树状数组数量
单点修改 + 子矩阵查询	直接维护二维前缀和	1
区间修改 + 单点查询	维护二维差分数组	1
区间修改 + 子矩阵查询	二维差分 + 数学推导，维护四个信息	4